Cards.py

Student worksheet - build card and deck primitives

Create a file called cards.py. Work through the steps in order. Each part adds one small piece to the final program. The final version should be able to create a shuffled deck and draw cards from it.

Before you start: Use meaningful names and keep the code readable. Test after each section instead of writing the whole file first.

0. Set up the file

At the top of cards.py, add the imports you will need.

```
from dataclasses import dataclass
from enum import IntEnum
from random import shuffle
```

Hint:
IntEnum lets enum values behave like integers
dataclass saves you from writing a long __init__ method.

1. create the class Suit(IntEnum):

Inside the class:

- initialise CLUBS, DIAMONDS, HEARTS, SPADES
- set them to 0, 1, 2, 3 respectively
- create a @property function called symbol
- return the initials: C, D, H, S

```
class Suit(IntEnum):
    CLUBS = 0
    # continue here
```

Hint: The property should not take extra parameters. It only needs self.

```
@property
def symbol(self) -> str:
    return {
        # map each Suit to its one-letter symbol
    }[self]
```

Checkpoint:

after this part, this should work:

```
print(Suit.CLUBS.symbol)  # expected: C
```

2. create the class Rank(IntEnum):

Inside the class:

- initialise the different ranks TWO, THREE, FOUR, ..., KING, ACE
- set them to 2, 3, 4, ..., 13, 14 respectively
- create a @property function called label
- return display labels such as 2, 3, ..., T, J, Q, K, A

```
class Rank(IntEnum):
    TWO = 2
    THREE = 3
    # continue until ACE = 14
```

Small reminder: For playing cards, Ten is usually written as T so that every card has a two-character form, for example TH for Ten of Hearts.

```
@property
def label(self) -> str:
    return {
        Rank.TWO: "2",
        # continue the mapping
    }[self]
```

Checkpoint:

```
print(Rank.ACE.label)  # expected: A
print(Rank.TEN.label)  # expected: T
```

3. create the class Card:

This class represents one single playing card.

- use @dataclass(frozen=True) above the class
- add two attributes: rank and suit
- rank should be of type Rank
- suit should be of type Suit
- create a __str__ method that returns rank label + suit symbol

```
@dataclass(frozen=True)
class Card:
    rank: Rank
    # continue here

    def __str__(self) -> str:
        # combine the rank label and suit symbol
        ...
```

Hint: If the card is Ace of Hearts, rank.label is A and suit.symbol is H. The final string should be AH.

Checkpoint:

```
card = Card(Rank.ACE, Suit.HEARTS)
print(card)  # expected: AH
```

4. create the class Deck:

This class stores the remaining cards in the deck.

- create an __init__(self) method
- inside __init__, create all 52 card combinations
- store the cards in self._cards
- shuffle self._cards after creating it

```
class Deck:
    def __init__(self) -> None:
        self._cards = [
            # create every Card(rank, suit) combination
        ]
        shuffle(self._cards)
```

Hint: You need two loops: one over Suit and one over Rank

Checkpoint:

```
deck = Deck()
print(len(deck._cards))  # expected: 52
```

5. create the method draw(self, count=1) (inside class Deck):

This method should remove cards from the deck and return them as a list.

- count should default to 1
- if count is less than 1, raise ValueError
- if count is larger than the number of remaining cards, raise ValueError
- otherwise remove count cards from self._cards and return them

```
def draw(self, count: int = 1) -> list[Card]:
    if count < 1:
        raise ValueError("count must be at least 1")
    ...
```

Why pop?: pop() removes one card from the list. This means the same card cannot be drawn twice.

6. final test

At the bottom of the file, or in a separate test file, try this:

```
deck = Deck()
hand = deck.draw(2)
print(hand)
print(len(deck._cards))   # expected: 50
```

Note: The exact cards will be different each time because the deck is shuffled.

Common mistakes to check

- Forgetting to write Suit.CLUBS instead of just CLUBS inside the dictionary.
- Returning the whole dictionary instead of indexing it with [self].
- Using shuffle(self._cards) before self._cards exists.
- Using pop without parentheses. It should be self._cards.pop().
- Returning one Card directly instead of a list of Card objects.

Optional extension

- Add a method remaining(self) that returns how many cards are left.
- Add a method reset(self) that creates a fresh shuffled deck.